AmigoBot with ROS 1

M. J. Khojasteh

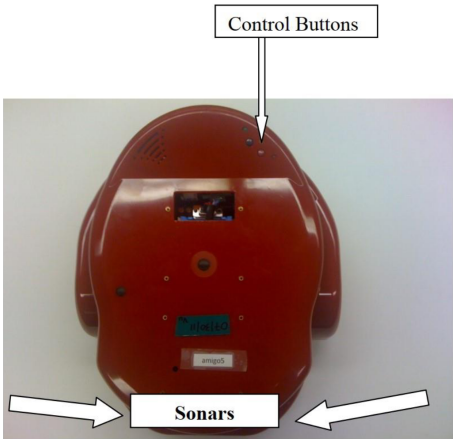
October 2026

1 Objective

In this experiment, students will use the AmigoBot by MobileRobots to investigate key aspects of mobile robot navigation in ROS 1 Noetic. They will design, simulate, and implement basic navigation algorithms, gaining hands-on experience throughout the process. The experiment also examines the positional errors that arise when using dead reckoning techniques in combination with encoder data.

Upon completion of this lab, students should be able to:

1. Use ROS publishers and subscribers to control a differential drive robot and read its sonar and odometry data.
2. Develop and test motion commands in simulation, then use the same ROS interface on the real robot.
3. Estimate the robot's pose from wheel velocities and evaluate accumulated dead reckoning error.



Technical Specifications:

Max forward/backward speed: 1 m/s;
Rotation speed: 100°/s;
Ultrasonic sensors: 1 on each side, 4 in front, 2 in the rear;
Encoders: 9,550 ticks per wheel revolution;
Microcontroller: 44 MHz Renesas SH2-7144;
Communication ports: 2× RS-232;
Control buttons: Red (Reset), Black (Motor Test).

2 Theory

Dead Reckoning

Dead reckoning, derived from the term "deduced reckoning" used in navigation, is a method of estimating an object's current position based on its previous location and known or estimated values of speed, time, or direction of travel. Because positional errors can accumulate over time, dead reckoning is often combined with other navigation techniques to improve accuracy and minimize cumulative error.

Encoders

Optical encoders, like those used in the AmigoBot, are commonly employed to measure the speed and position of a wheel shaft. They consist of a light source, a fixed grating to modulate the light, a rotating disc with a fine optical pattern attached to the wheel shaft, and stationary optical detectors. As the shaft rotates, the detectors sense varying light levels depending on the alignment of the fixed and rotating gratings.

The AmigoBot provides 9,550 encoder ticks per wheel revolution (approximately 30 ticks per millimetre of travel). Quadrature encoders use two signals offset by 90 degrees to determine shaft movement and direction. In this lab, the provided ROS interface supplies wheel angular velocities in radians per second, so your odometry node does not need to decode raw encoder ticks.

3 Introduction

The AmigoBot is a compact, intelligent, two-wheeled differential drive mobile robot and the smallest model in the Pioneer robot family developed by MobileRobots Inc. It is widely used by research institutions around the world for robotics education and experimentation. Designed for autonomous navigation in real-world environments, the AmigoBot is equipped with essential components including a rechargeable battery, drive motors, two main wheels with a supporting caster wheel, position and speed encoders, sonar range sensors, and an onboard microcontroller.

ROS interface with AmigoBot

- The supplied workspace contains five packages: `amigo_description` (robot model and parameters), `amigo_gazebo` (simulation), `amigo_bringup` (real robot), `amigo_tools` (interface and test tools), and `amigo_student` (task skeletons).
- The container includes ROS 1 Noetic, Gazebo 11, ROSARIA and the ARIA robot library. The supplied packages replace the older MABL examples; no additional repository clones or launch-file fixes are needed.
- Simulation and hardware provide the same command, sonar, odometry and wheel-velocity topics. Only `/ground_truth/state` is exclusive to simulation.

Workspace Setup

1. Install Docker (Docker Desktop on macOS or Windows, or Docker Engine on Linux) and VS Code with the **Dev Containers** extension.
2. Open the supplied `lab5/` folder itself in VS Code. Run **Dev Containers: Reopen in Container** and wait for the first build to finish. The container builds `catkin_ws` automatically.
3. Open `http://localhost:6080` in your browser for the desktop where Gazebo and RViz appear. No separate X server is required.
4. Run all ROS commands below in VS Code terminals *inside the container*. New terminals start in `~/catkin_ws` with ROS and the workspace already sourced.

Edit the Python skeletons in `amigo_student/scripts/` under `lab5/catkin_ws/src/amigobot_ros/`. They run but do not implement the tasks until you fill in their TODOs. Python edits need no rebuild. If you add a Python script, make it executable:

Terminal

```
chmod +x ~/catkin_ws/src/amigobot_ros/amigo_student/scripts/my_new_node.py
```

For new packages or compiled-code changes, run `catkin_make` in `~/catkin_ws`. Then run `source devel/setup.bash` in each terminal using it.

Launching the Simulation

In one container terminal, run:

Terminal

```
roslaunch amigo_gazebo amigo_sim.launch
```

Gazebo and RViz open on the browser desktop. Leave this terminal running. In another terminal, check the interface:

Terminal

```
roslaunch amigo_tools verify.launch
```

This check drives about 0.5 m and turns the robot. Before Task 4, expect 25 passed checks, 0 failures and one warning that `/my_odom` is missing. That topic is your implementation in Task 4. **Restart the simulator before each task** so the robot starts at the origin with clear space for the test patterns.

Running on the Real Robot

Test your code in simulation first. For physical-robot check-offs, obtain the current bridge address and network instructions from the TA. **The old `amigoN.rit.edu` hostnames no longer resolve.** Physical-robot connectivity is still being restored; the software path has been tested only against a protocol simulator.

Stop the Gazebo launch before starting the real-robot launch. Connect your computer to the robot's network, set `ROBOT_IP` to the address provided by the TA, and run:

Terminal

```
roslaunch amigo_bringup amigo_real.launch robot_ip:=$ROBOT_IP
```

The bridge uses TCP port 8101 by default; add `tcp_port:=PORT` if the TA specifies another port. After connecting, check the stationary interface before attempting motion:

Terminal

```
roslaunch amigo_tools verify.launch hardware:=true move:=false
```

Use a low speed (for example, 0.10 m/s) for physical-robot demonstrations. The same student nodes use the same topics in simulation and on hardware.

4 Lab Tasks

Task 1: Run AmigoBot in Simulation

- Launch the simulator as described above and demonstrate teleoperation to the TA.
- For keyboard control, run the following in a separate terminal. Keep that terminal focused; `i` drives forward and `k` stops.

Terminal

```
rosrun teleop_twist_keyboard teleop_twist_keyboard.py
```

Alternatively, launch the simulator with `steering:=true` to open the **Robot Steering** sliders:

Terminal

```
roslaunch amigo_gazebo amigo_sim.launch steering:=true
```

Use one teleoperation method at a time. The slider window continuously publishes its current command, including zero, and will override keyboard commands. Close it before running a motion program.

Task 2: Write a Publisher to Move the Robot

- Complete `turn_circle.py` in `amigo_student`, or implement an equivalent C++ node.
- Publish `geometry_msgs/Twist` on `/cmd_vel` to drive a circle. For speed v and radius R, set `linear.x` to v and `angular.z` to v/R.
- Read the speed and radius from ROS parameters. Publish continuously (for example, at 20 Hz), and publish a zero `Twist` on shutdown. Keep speeds at or below 0.2 m/s indoors.
- Demonstrate in simulation and, when the TA confirms availability, on hardware. Use the same topic names and motion code.

Terminal

```
rosrun amigo_student turn_circle.py _speed:=0.15 _radius:=0.5
```

Stop your node before starting another command publisher. If the robot continues moving after a node exits, send a stop command:

Terminal

```
rosrun amigo_tools drive_pattern.py _pattern:=stop
```

Task 3: Subscribe to Sonar and Odometry

- Complete `sensor_listener.py`, or an equivalent C++ node. Subscribe to `/sonar` (`sensor_msgs/PointCloud`) and `/odom` (`nav_msgs/Odometry`). **Sonar works in simulation and on hardware.**
- Read the cloud's `index` and `range` channels to associate each transducer with its range in metres. Alternatively, individual readings are available as `sensor_msgs/Range` on `/sonar/range_0` through `/sonar/range_7`.
- Sonar numbering is: 0 left side, 1–4 across the front, 5 right side, and 6–7 rear.
- Display the eight ranges and the robot's x, y and yaw at a readable rate (the skeleton uses 2 Hz). Demonstrate the subscriber to the TA.

Terminal

```
rosrun amigo_student sensor_listener.py
```

Task 4: Estimate Odometry from Wheel Velocities

Complete `my_odom.py` to subscribe to `/wheel_velocities` (`std_msgs/Float32MultiArray`) and publish `nav_msgs/Odometry` on `/my_odom`. Use differential drive kinematics and observe these conventions:

- `msg.data[0]` is the left wheel angular velocity and `msg.data[1]` is the right wheel angular velocity, both in rad/s.
- Read the parameters `~wheel_radius` and `~wheel_separation`. The launch file loads them from `amigo_description/config/amigo_params.yaml`. The wheel radius is 0.0508 m (diameter 101.6 mm). The configured wheel separation is 0.280 m; this remains an estimate awaiting physical calibration.
- Set `header.frame_id` to `odom` and `child_frame_id` to `base_link`. Fill in the timestamp, pose and twist. **Do not broadcast another `odom` to `base_link` TF transform**; the driver already publishes it.
- Initialize x, y and heading θ to zero. Since `Float32MultiArray` has no timestamp, obtain each callback's time from `rospy.Time.now()` (or `ros::Time::now()` in C++). Use the first sample to initialize timing, then compute Δt from consecutive times and skip nonpositive intervals. Do not hard-code Δt from the nominal publishing rate.

Let r be the wheel radius, L the wheel separation, and ω_L, ω_R the wheel angular velocities. For each interval:

$$\begin{aligned} v_L &= r\omega_L, & v_R &= r\omega_R, \\ d_L &= v_L \Delta t, & d_R &= v_R \Delta t, \\ d &= \frac{d_R + d_L}{2}, & \Delta\theta &= \frac{d_R - d_L}{L}. \end{aligned}$$

Using midpoint integration, update the pose as follows:

$$\begin{aligned} x_{k+1} &= x_k + d\cos(\theta_k + \Delta\theta/2), \\ y_{k+1} &= y_k + d\sin(\theta_k + \Delta\theta/2), \\ \theta_{k+1} &= \theta_k + \Delta\theta. \end{aligned}$$

Here v_L, v_R are linear velocities in m/s, while d_L, d_R, d are distances in metres. Publish body twist $v = d/\Delta t$ and $\omega = \Delta\theta/\Delta t$. Convert yaw to the quaternion $(0, 0, \sin(\theta/2), \cos(\theta/2))$ for the orientation field. You may instead use Euler integration (projecting with θ_k); state which method you used. The methods differ on arcs, so use a circle if you compare them; a square of straight segments and in-place turns does not expose that difference.

Explore `/odom` to inspect the message structure. In RViz, add an **Odometry** display for `/my_odom`. To inspect the TF tree, run `rosrun rqt_tf_tree rqt_tf_tree`.

Test and measure the error

Start from a freshly launched simulator. In a second terminal, run your node together with the supplied error logger:

Terminal

```
roslaunch amigo_student my_odom.launch
```

Wait for your odometry messages, then, in a third terminal, drive two 1.5 m squares (about two minutes):

Terminal

```
rosrun amigo_tools drive_pattern.py _pattern:=square _side:=1.5 _laps:=2
```

When the drive finishes, stop `my_odom.launch` with `Ctrl-C`. The logger compares your estimate with `/ground_truth/state`, prints final and peak position errors and final heading error, and writes `~/amigo_odom_error.csv` when it stops. Use these measurements to evaluate accumulated error. With your node running, you can also rerun `roslaunch amigo_tools verify.launch` to check the interface.

On the real robot, select its own odometry as the reference:

Terminal

```
roslaunch amigo_student my_odom.launch reference:=/odom
```

On hardware, the supplied wheel velocities are reconstructed from the body twist in `/odom`; they are not independent per-wheel measurements. Agreement with `/odom` therefore checks consistency, not physical accuracy. Compare against an external position measurement when assessing real-robot drift.

Extra Credit

Complete `wander.py` to implement random wandering with sonar-based obstacle avoidance. Test in simulation first, then demonstrate on the real robot when available. Use the front sonar to stop or turn away from obstacles, check rear sonar before reversing, and publish a zero command on shutdown. Real sonar can miss angled surfaces and produce isolated short readings; the simulator does not model these effects. Stay near the physical robot while it runs.

Terminal

```
rosrun amigo_student wander.py _speed:=0.15 _stop_distance:=0.45
```

5 Report

1. Submit a technical memo. A skeleton/guideline is available on MyCourses.
2. Submit all materials electronically via MyCourses.
3. Your submission must be packaged into a single `.zip` archive.
4. Submit a hard copy of your signed rubric to the TA (either in person or in their mail folder).
    - The lab is not considered fully submitted or on time until **both** of the following are submitted:
        - A `.zip` archive containing the lab report and code.
        - A hard copy of the signed rubric.
5. Late submissions will receive a 10% penalty per day late.
    - The first 10% penalty will be applied at 12:01 a.m. the day after the demo/report is due.
6. Include the following items specific to this lab:
    - A description of the theory of dead reckoning.
    - An explanation of how sonar works.